Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №7
Принципы SOLID и рефакторинг архитектуры Python-приложения

1. Цель работы

Получить практические навыки проектирования и рефакторинга объектно-ориентированных приложений на Python с использованием принципов SOLID.

2. Постановка задачи

Разработать систему управления номерами и бронированиями для гостиницы (Вариант 5). Операции: бронирование, заселение, выселение, расчёт стоимости. Необходимо отделить: управление бронированием (бизнес-логика), хранение данных (репозиторий), уведомление клиента (инфраструктура), формирование отчётов (сервис отчетов). Работа состоит из двух этапов: создание проблемной архитектуры и последующий рефакторинг.

3. Этап 1. Реализация проблемной архитектуры

На первом этапе была создана рабочая версия приложения, содержащая намеренные архитектурные проблемы для демонстрации необходимости применения SOLID. Класс HotelManager совмещал хранение данных, бизнес-логику, уведомления, отчёты и вывод в консоль — единая ответственность нарушена (SRP); добавление SMS-уведомления требовало изменения класса (OCP); отсутствовали абстракции (LSP, ISP); клиентский код жёстко зависел от конкретного класса и сам создавал зависимости (DIP).

Принцип	Нарушение в проблемной версии
SRP	HotelManager отвечает за хранение, логику, уведомления, отчёты, консоль
OCP	новый способ уведомления требует изменения book_room
LSP	нет абстракций, подмена реализаций невозможна
ISP	нет интерфейсов, класс перегружен методами
DIP	зависимости создаются внутри класса, main жёстко связан

4. Этап 2. Рефакторинг архитектуры с соблюдением SOLID

Приложение разделено на логические слои: models (сущности и перечисления), interfaces (Protocol Repository и Notifier), repositories (InMemoryRepository), services (HotelService, ReportService), infrastructure (уведомители), ui (ConsoleUI) и main (композиционный корень).

```
lab_7/
├── main.py                     # композиционный корень (DIP)
├── models/
│   ├── entities.py             # Room, Guest, Booking (только данные)
│   └── enums.py                # RoomStatus
├── interfaces/
│   ├── repository.py           # Protocol: add/get/get_all/update
│   └── notifier.py             # Protocol: send
├── repositories/
│   └── memory_repository.py    # InMemoryRepository (LSP)
├── services/
│   ├── domain_service.py       # HotelService — бизнес-логика (SRP)
│   └── report_service.py       # ReportService — отчёты (SRP)
├── infrastructure/
│   └── notifiers.py            # Email/Sms/ConsoleNotifier (OCP)
└── ui/
    └── console.py              # только ввод/вывод (SRP)
```

Файл: interfaces/repository.py и notifier.py — маленькие интерфейсы (ISP)

```
from typing import Protocol, List, Any

# ISP: Разделение интерфейсов. Репозиторий умеет только хранить данные.
class Repository(Protocol):
    def add(self, entity: Any) -> None: ...
    def get(self, entity_id: int) -> Any: ...
    def get_all(self) -> List[Any]: ...
    def update(self, entity: Any) -> None: ...
```

```
from typing import Protocol

# ISP: Отдельный маленький интерфейс для уведомлений.
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None: ...
```

Файл: repositories/memory_repository.py — реализация (LSP)

```
from typing import List, Any
from interfaces.repository import Repository

# LSP: Реализация репозитория в памяти. Может быть заменена на FileRepository
без изменения сервиса.
class InMemoryRepository(Repository):
    def __init__(self):
        self._data: dict[int, Any] = {}

    def add(self, entity: Any) -> None:
        self._data[entity.id] = entity

    def get(self, entity_id: int) -> Any:
        return self._data.get(entity_id)

    def get_all(self) -> List[Any]:
        return list(self._data.values())

    def update(self, entity: Any) -> None:
        if entity.id in self._data:
            self._data[entity.id] = entity
```

Файл: services/domain_service.py — бизнес-логика с DI (SRP, DIP)

```
from models.entities import Room, Booking
from models.enums import RoomStatus
from interfaces.repository import Repository
```

```
from interfaces.notifier import Notifier

# SRP: Сервис отвечает только за бизнес-логику бронирования.
# DIP: Зависимости (репозитории и уведомитель) передаются извне через
конструктор (Dependency Injection).
class HotelService:
    def __init__(self, room_repo: Repository, booking_repo: Repository,
notifier: Notifier):
        self.room_repo = room_repo
        self.booking_repo = booking_repo
        self.notifier = notifier

    def book_room(self, room_id: int, guest_name: str, days: int) -> Booking:
        room = self.room_repo.get(room_id)
        if not room:
            raise ValueError("Номер не найден")

        if room.status != RoomStatus.FREE:
            raise ValueError(f"Номер {room.number} недоступен (статус:
{room.status.value})")

        room.status = RoomStatus.BOOKED
        self.room_repo.update(room)

        cost = room.price_per_night * days
        booking = Booking(
            id=len(self.booking_repo.get_all()) + 1,
            room_id=room.id,
            guest_name=guest_name,
            days=days,
            total_cost=cost
        )
        self.booking_repo.add(booking)

        self.notifier.send(guest_name, f"Вы забронировали номер {room.number} на
{days} дней. Стоимость: {cost} руб.")
        return booking

    def check_in(self, room_id: int) -> None:
        room = self.room_repo.get(room_id)
        if not room:
            raise ValueError("Номер не найден")
        if room.status != RoomStatus.BOOKED:
            raise ValueError(f"Нельзя заселиться: номер {room.number} не
забронирован")

        room.status = RoomStatus.OCCUPIED
        self.room_repo.update(room)
        self.notifier.send("Администратор", f"Гость заселен в номер
{room.number}")

    def check_out(self, room_id: int) -> None:
        room = self.room_repo.get(room_id)
        if not room:
            raise ValueError("Номер не найден")
        if room.status != RoomStatus.OCCUPIED:
            raise ValueError(f"Нельзя выселиться: номер {room.number} не занят")

        room.status = RoomStatus.FREE
        self.room_repo.update(room)
        self.notifier.send("Администратор", f"Гость выселен из номера
{room.number}. Номер свободен.")
```

Файл: infrastructure/notifiers.py — расширение без изменения сервиса (OCP)

```
from interfaces.notifier import Notifier

class EmailNotifier(Notifier):
    def send(self, recipient: str, message: str) -> None:
        print(f"[EMAIL to {recipient}]: {message}")

class SmsNotifier(Notifier):
```

```
    def send(self, recipient: str, message: str) -> None:
        print(f"[SMS to {recipient}]: {message}")

class ConsoleNotifier(Notifier):
    def send(self, recipient: str, message: str) -> None:
        print(f"[УВЕДОМЛЕНИЕ для {recipient}]: {message}")
```

Файл: main.py — композиционный корень (DIP)

```
from repositories.memory_repository import InMemoryRepository
from infrastructure.notifiers import ConsoleNotifier
from services.domain_service import HotelService
from services.report_service import ReportService
from ui.console import ConsoleUI

def main():
    # DIP: Внедрение зависимостей (Dependency Injection) через конструктор
    room_repo = InMemoryRepository()
    booking_repo = InMemoryRepository()
    notifier = ConsoleNotifier() # Легко заменить на EmailNotifier() или
SmsNotifier()

    # Инициализация сервисов
    hotel_service = HotelService(room_repo, booking_repo, notifier)
    report_service = ReportService(room_repo, booking_repo)

    # Запуск UI
    ui = ConsoleUI(hotel_service, report_service)
    ui.run()

if __name__ == "__main__":
    main()
```

5. Демонстрация соблюдения принципов SOLID

Демонстрация (рисунок 7.1): бронирование номера через сервис с внедрёнными зависимостями, заселение и выселение, подмена уведомителя (Console → Email → SMS) без изменения HotelService, формирование отчёта отдельным сервисом.

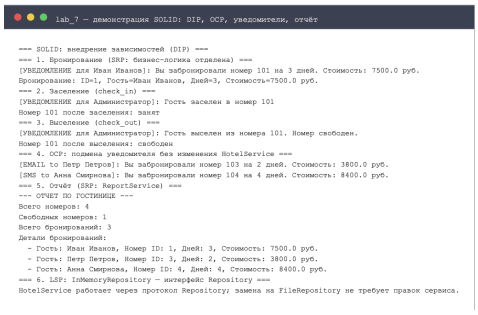


Рисунок 7.1 – Демонстрация: DIP, OCP, SRP, уведомители, отчёт

6. Тестирование

Написаны 17 автоматических тестов (pytest): репозиторий (add/get/get_all/update), сервисный слой (бронирование, заселение, выселение, проверки статусов и ошибок), отчёты, а также проверка замены уведомителей (OCP/LSP). Все тесты прошли успешно (рисунок 7.2).

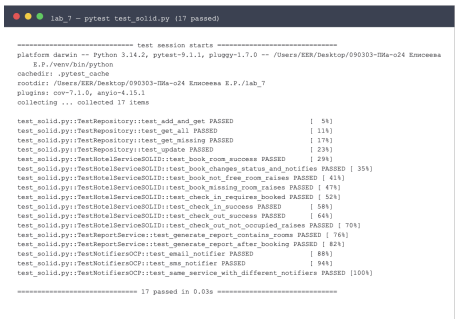


Рисунок 7.2 – Результаты pytest (17 passed)

7. Вывод

В ходе выполнения лабораторной работы были изучены принципы проектирования SOLID. На первом этапе разработана монолитная архитектура, наглядно демонстрирующая проблемы жёсткой связанности и множественных ответственностей. На втором этапе проведён рефакторинг: приложение разделено на логические слои, применены Dependency Injection, маленькие интерфейсы (Protocol) и паттерн Repository. Это позволило сделать систему гибкой, расширяемой (новые уведомители добавляются без изменения сервиса) и легко тестируемой.